JavaScript

How to - Become a Front-End Developer

A Front-End Developer is someone who creates websites and web applications. The Front-End Developer creates things that the user sees. It is a popular job, and everyone can become a Front-End Developer.

What does a Front-End Developer do?

The main responsibility of a Front-End Developer is the **User interface**. Simply put, create things that the user sees.

The difference between Front-End and Back-End is that Front-End refers to how a web page looks, while back-end refers to how it works. You also think of Front-End as **client-side** and Back-End as **server-side**.

JavaScript is the world's most popular programming language. JavaScript is the programming language of the Web.

What is JavaScript?

JavaScript is the programming language of the web. It can update and change both HTML and CSS. It can calculate, manipulate and validate data.

Why Study JavaScript?

JavaScript is one of the 3 languages all web developers must learn:

1. **HTML** to **Create the structure with HTML.** The first thing you have to learn is HTML, which is the standard mark-up language for creating web pages.

2. CSS to Style with CSS. The next step is to learn CSS, to set the layout of your web page with beautiful colours, fonts, and much more.

3. JavaScript to program the behaviour of web pages and make it interactive with JavaScript. After studying HTML and CSS, you should learn JavaScript to create dynamic and interactive web pages for your users.

JavaScript Can Change HTML Content

One of many JavaScript HTML methods is `getElementById()`.

The example below "finds" an HTML element (with id="par"), and changes the element content (innerHTML) to "Hello JavaScript":

```
document.getElementById("demo").innerHTML = "Hello JavaScript";
```

< html>

<html>

<body>

<h2>What Can JavaScript Do?</h2>

JavaScript can change HTML content.

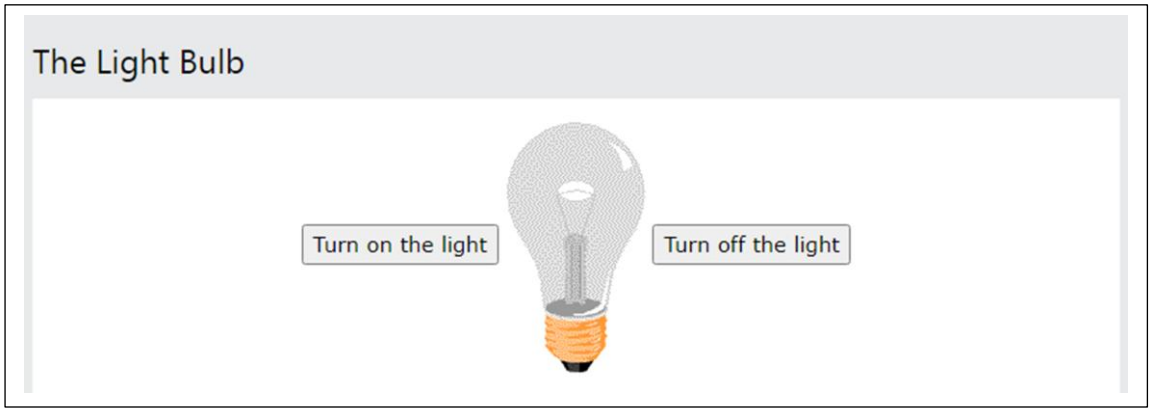
<button type="button" onclick='document.getElementById("par").innerHTML = "Hello JavaScript!"'>Click Me!</button>

</body>

</html>

JavaScript Can Change HTML Attribute Values

In this example JavaScript changes the value of the src (source) attribute of an <img> tag:



<html>

<body>

<h2>What Can JavaScript Do?</h2>

JavaScript can change HTML attribute values.

In this case JavaScript changes the value of the src (source) attribute of an image.

<button onclick="document.getElementById('myImage').src='b2.jpg'">Turn on the light</button>

<img id="myImage" src="b1.jpg" style="width:100px">

<button onclick="document.getElementById('myImage').src='b1.jpg'">Turn off the light</button>

</body>

```
<html>
<head>
<script>
function myFunction() {
  document.getElementById("demo").innerHTML = "Paragraph changed.";
}
</script>
</head>
<body>
<h2>Demo JavaScript in Head</h2>
<p id="demo">A Paragraph.</p>
<button type="button" onclick="myFunction()">Try it</button>
</body>
</html>
```

Window Console Object

The Console Object

The **console object** provides access to the browser's debugging console.

The **console object** is a property of the **window object**.

The **console object** is accessed with:

`window.console` or just `console`

Example 1

```
<html>

<body>

<h1>The Console Object</h1>

<h2>The error() Method</h2>

<p>Remember to open the console (Press F12) before you click "Run".</p>

<script>

console.error("You made a mistake");

</script>

</body>

</html>
```

Console Object Methods

Method	Description
assert()	Writes an error message to the console if a assertion is false
clear()	Clears the console
count()	Logs the number of times that this particular call to count() has been called
error()	Outputs an error message to the console
group()	Creates a new inline group in the console. This indents following console messages by an additional level, until console.groupEnd() is called
groupCollapsed()	Creates a new inline group in the console. However, the new group is created collapsed. The user will need to use the disclosure button to expand it
groupEnd()	Exits the current inline group in the console
info()	Outputs an informational message to the console
log()	Outputs a message to the console

table()	Displays tabular data as a table
time()	Starts a timer (can track how long an operation takes)
timeEnd()	Stops a timer that was previously started by console.time()

JavaScript Error message

```
<html>

<body>

<h1>JavaScript Errors</h1>

<p>In this example we have written "alert" as "adddlert" to deliberately produce an error.</p>

<p>The name property of the Error object returns the name of the error and the message property returns a description of the error:</p>

<p id="demo" style="color:red"></p>

<script>

try {

  addalert("Welcome guest!");

}

catch(err) {

document.getElementById("demo").innerHTML = err.message;

}

</script>

</body>
```

JavaScript Variables

Variables are Containers for Storing Data

JavaScript Variables can be declared in 4 ways:

- Automatically

They are automatically declared when first used:

```
x = 5;
y = 6;
z = x + y;
```

- Using `var`

```
var x = 5;
var y = 6;
var z = x + y;
```

- Using `let`

```
let x = 5;
let y = 6;
let z = x + y;
```

- Using `const`

```
const x = 5;
const y = 6;
const z = x + y;
```

When to Use var, let, or const?

1. Always declare variables

2. Always use const if the value should not be changed

3. Always use const if the type should not be changed (Arrays and Objects)

4. Only use let if you can't use const

5. Only use var if you MUST support old browsers.

Example 1

<html>

```
<body>
<h1>JavaScript Variables</h1>
<p>In this example, x, y, and z are undeclared.</p>
<p>They are automatically declared when first used.</p>
<p id="demo"></p>
<script>
x = 5;
y = 6;
z = x + y;
document.getElementById("demo").innerHTML ="The value of z is: " + z;
</script>
</body>
</html>
```

Example2

```
<html>
<body>
<h1>JavaScript Variables</h1>
<p>Create a variable, assign a value to it, and display it:</p>
<p id="demo"></p>
<script>
let carName = "Volvo";
document.getElementById("demo").innerHTML = carName;
</script>
</body>
```

</html>

One Statement, Many Variables

let person = "John Doe", carName = "Volvo", price = 200;

JavaScript Arithmetic Operators

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation s
/	Division
%	Modulus (Division Remainder)
++	Increment
--	Decrement

Operator	Example	Same As
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y
**=	x **= y	x = x ** y

JavaScript Comparison Operators

Operator	Description
==	equal to
===	equal value and equal type
!=	not equal
!==	not equal value or not equal type

>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
?	ternary operator

Example3

```
<html>
<body>
<h1>JavaScript String Operators</h1>
<h2>The += Operator</h2>
<p>The assignment operator += can concatenate strings.</p>
<p id="demo"></p>
<script>
let text1 = "What a very ";
text1 += "nice day";
document.getElementById("demo").innerHTML = text1;
</script>
</body> </html>
```

Example4

```
<html>
```

```
<body>

<h1>JavaScript String Operators</h1>

<h2>The += Operator</h2>

<p>The assignment operator += can concatenate strings input from text1 and text2.</p>

<p id="demo"></p>

<input type="text" id="t1" >

<input type="text" id="t2" >

<button type="button" onclick="myFunction()">Test Input</button>

<script>

function myFunction() {

  let x1 = document.getElementById("t1").value;

  let x2 = document.getElementById("t2").value;

x1 += x2;

document.getElementById("demo").innerHTML = x1;

}

</script>

</body> </html>
```

JavaScript Logical Operators

Operator	Description
&&	logical and
||	logical or

!	logical not

JavaScript has 8 Data types

String
Number
Bigint
Boolean
Undefined
Null
Symbol
Object

JavaScript Arrays

An array is a special variable, which can hold more than one value:

```
const cars = ["Saab", "Volvo", "BMW"];
```

Example

<html>

<body>

<h1>JavaScript Arrays</h1>

<script>

const cars = ["Ford", "Volvo", "BMW"];

document.getElementById("demo").innerHTML = cars;

</script>

</body>

</html>

JavaScript Objects

JavaScript object is a variable that can store multiple data in key-value pairs.

Example

```
// student object
<script>
const student = {
        firstName: "Jack",
        rollNo: 32,
    };
    student.rollNo=35;
    console.log(student);
//Delete object properity
    delete student.rollNo;
    console.log(student);
</script>
```

JavaScript for Loop

Loops are handy, if you want to run the same code over and over again, each time with a different value.

Often this is the case when working with arrays:

Example

```
<html>
<body>
```

```
<h2>JavaScript For Loop</h2>

<p id="demo"></p>

<script>

const cars = ["BMW", "Volvo", "Saab", "Ford", "Fiat", "Audi"];

let text = "";

for (let i = 0; i < cars.length; i++) {

  text += cars[i] + "<br>";

}

document.getElementById("demo").innerHTML = text;

</script>

</body>

</html>
```

Conditional Statements

Very often when you write code, you want to perform different actions for different decisions.

You can use conditional statements in your code to do this.

In JavaScript we have the following conditional statements:

JavaScript if, else, and else if

- Use `if` to specify a block of code to be executed, if a specified condition is true
- Use `else` to specify a block of code to be executed, if the same condition is false
- Use `else if` to specify a new condition to test, if the first condition is false
- Use `switch` to specify many alternative blocks of code to be executed

```
if (condition) {
```

```
  //  block of code to be executed if the condition is true
} else {
  //  block of code to be executed if the condition is false
}
```

Example

```
<html>
<body>
<h2>JavaScript Math.random()</h2>
<p id="demo"></p>
<script>
let text;
if (Math.random() < 0.5) {
  text = "<a href='https://w3schools.com'>Visit W3Schools</a>";
} else {
  text = "<a href='https://wwf.org'>Visit WWF</a>";
}
document.getElementById("demo").innerHTML = text;
</script>
</body>
</html>
```